TP 1 : Programmation et transactions

Programmation PL/pgSQL, procédures, triggers et concurrence

Module : Bases de Données Avancées
Niveau : M1
Responsable du TP : M. Alachaher
Année universitaire : 2026/2027

Objectif

Apprendre à programmer dans PostgreSQL avec PL/pgSQL, automatiser des traitements métier avec des fonctions, procédures et triggers, puis garantir la cohérence des données grâce aux transactions. Chaque partie correspond à une séance.

Préparation

Exécuter le fichier `dataset_tp1.sql` dans le *Query Tool* de pgAdmin, dans une base de données de travail **Specilaité_numGroupe**. Le script crée le schéma `tp1_banque` et les tables suivantes :

Table	Rôle et attributs
`client`	`id_client`, `nom`, `prenom`, `ville` et `date_inscription`.
`compte`	`id_compte`, `id_client`, `type_compte`, `solde`, `date_ouverture` et `actif`.
`operation`	`id_operation`, `id_compte`, `type_operation`, `montant`, `date_operation` et `commentaire`.
`historique_solde`	`id_historique`, `id_compte`, `ancien_solde`, `nouveau_solde` et `date_modification`. Cette table est initialement vide.

Le script contient 8 clients, 10 comptes et 10 opérations historiques. Ces opérations sont **déjà intégrées aux soldes initiaux** : elles ne doivent pas être rejouées. Avant chaque nouvelle séance, réexécuter le script si vous souhaitez repartir de l'état initial.

Partie 1 — Programmation procédurale avec PL/pgSQL

Objectif

Utiliser des variables, des conditions, des boucles et créer une première fonction PL/pgSQL.

Rappel syntaxe / Exemple

Un bloc `DO` permet d'exécuter un traitement procédural sans créer de fonction permanente. La partie `DECLARE` contient les variables, `BEGIN ... END` les instructions, et `RAISE NOTICE` affiche un message.

```
DO $$
DECLARE
    montant NUMERIC = 12000;
BEGIN
    IF montant >= 10000 THEN
        RAISE NOTICE 'Montant important : %', montant;
    ELSE
        RAISE NOTICE 'Montant ordinaire';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Une fonction retourne un résultat et peut être appelée dans un `SELECT` :

```
CREATE OR REPLACE FUNCTION doubler(valeur NUMERIC)
RETURNS NUMERIC AS
$$
BEGIN
    RETURN valeur * 2;
END;
$$ LANGUAGE plpgsql;

SELECT doubler(250);
```

Pour parcourir des valeurs : `FOR i IN 1..5 LOOP ... END LOOP;`.
Pour récupérer une valeur depuis une table et la stocker dans une variable, on utilise `SELECT ... INTO` La clause WHERE permet de sélectionner l'enregistrement souhaité.

Exercice 1 — Variables et conditions

1. Écrire un bloc `DO` qui déclare un montant de retrait de 7 000 DA et affiche s'il dépasse ou non 10 000 DA.
2. Modifier le bloc pour distinguer trois catégories : moins de 5 000 DA, de 5 000 à 20 000 DA inclus, et plus de 20 000 DA.
3. Tester les montants 4 000, 12 000 et 25 000 DA. Vérifier les messages affichés.

Exercice 2 — Boucles

1. Écrire un bloc qui affiche les entiers de 1 à 5 à l'aide d'une boucle `FOR`.
2. Modifier le bloc pour calculer la somme des entiers de 1 à 10.
3. Afficher le résultat final à l'aide de `RAISE NOTICE`.

Exercice 3 — Première fonction métier

1. Créer une fonction `calculer_bonus(montant NUMERIC)` qui retourne 2 % du montant reçu.
2. Tester la fonction avec 10 000 et 50 000 DA.

Exercice 4 — fonction avancee

1. Créer une fonction `est_solde_suffisant(id INTEGER, retrait NUMERIC)` retournant `BOOLEAN` selon que le compte possède un solde suffisant pour le retrait demandé.
2. Tester la fonction sur les comptes 101 et 106 avec plusieurs montants. Justifier les résultats.

Remarque

Une fonction de consultation ne doit pas modifier les soldes. En cas de réinitialisation du dataset, les fonctions créées dans le schéma seront également supprimées.

Partie 2 — Procédures et transactions

Objectif

Créer et appeler des procédures PL/pgSQL pour réaliser des opérations bancaires.

Rappel syntaxe / Exemple

Une **procédure** regroupe des instructions SQL et PL/pgSQL et s'exécute avec `CALL`. Contrairement à une fonction appelée par `SELECT`, elle est particulièrement adaptée aux traitements métier qui modifient les données.

```
CREATE OR REPLACE PROCEDURE afficher_solde(p_id INTEGER)
LANGUAGE plpgsql AS $$
DECLARE
    v_solde NUMERIC;
BEGIN
    SELECT solde INTO v_solde
    FROM compte WHERE id_compte = p_id;
    RAISE NOTICE 'Solde du compte % : % DA', p_id, v_solde;
END;
$$;

CALL afficher_solde(101);
```

Exercice 1 — Première procédure de consultation

1. Réexecuter le script de dataset_tp1.sql
2. Créer une procédure `afficher_compte(id_c INTEGER)` affichant, avec `RAISE NOTICE`, le solde d'un compte.
3. Tester la procédure avec les comptes 101 et 105.
4. Modifier la procédure pour afficher également le type de compte.
5. Prévoir un message explicite si le numéro de compte n'existe pas.

Exercice 2 — Procédure de dépôt

Créer une procédure `effectuer_depot(c_id INTEGER, d_montant NUMERIC)` qui :

1. Ajoute le montant au solde du compte indiqué.
2. Refuser un montant nul ou négatif et un compte inexistant.
3. Enregistrer le dépôt dans la table `operation`, avec le type `depot`.
4. Tester un dépôt de 3 000 DA sur le compte 104, puis un dépôt négatif. Comparer le solde et le journal des opérations avant et après chaque appel.

Exercice 3 — Procédure de virement

1. Créer une procédure `effectuer_virement(c_source INTEGER, c_destination INTEGER, v_montant NUMERIC)` qui débite un compte et crédite un autre.
2. Vérifier que les comptes existent, sont actifs, sont différents et que le montant est strictement positif.

3. Refuser le virement si le solde du compte source est insuffisant. Enregistrer deux opérations : `virement_sortant` et `virement_entrant`.

4. Tester un virement valide de 2 010 DA du compte 101 vers le compte 104, puis un virement de 100 000 DA du compte 110 vers le compte 103. Vérifier qu'un virement refusé ne modifie aucun solde.

Exercice 4 — Expérience guidée : transactions et concurrence (observation)

Cet exercice est fourni avec son code : **exécuter les blocs séparément**, observer les résultats et répondre aux questions.

A. Observer `ROLLBACK` et `COMMIT`. Relever d'abord le solde du compte 109. Exécuter le premier bloc, puis le second, en comparant les soldes avant, pendant et après chaque transaction.

```
BEGIN;
UPDATE compte SET solde = solde + 1000 WHERE id_compte = 109;
ROLLBACK;
SELECT solde FROM compte WHERE id_compte = 109;

BEGIN;
UPDATE compte SET solde = solde + 1000 WHERE id_compte = 109;
COMMIT;
SELECT solde FROM compte WHERE id_compte = 109;
```

Questions : Quelle modification a été annulée ? Laquelle a été conservée ?

B. Observer `SAVEPOINT`. Exécuter ce bloc en entier et comparer les soldes des comptes 107 et 109 avant et après.

```
BEGIN;
UPDATE compte SET solde = solde + 500 WHERE id_compte = 107;
SAVEPOINT etape1;
UPDATE compte SET solde = solde + 700 WHERE id_compte = 109;
ROLLBACK TO SAVEPOINT etape1;
COMMIT;
SELECT id_compte, solde FROM compte
WHERE id_compte IN (107, 109) ORDER BY id_compte;
```

Questions : Pourquoi une seule augmentation est-elle conservée ? Quel est le rôle du point de sauvegarde ?

C. Concurrence avec deux fenêtres du Query Tool. Ouvrir deux fenêtres **A** et **B**, connectées à la même base. Dans chacune, exécuter `SET search_path TO tp1_banque, public;`.

Fenêtre A :

```
BEGIN;
UPDATE compte SET solde = solde + 200 WHERE id_compte = 105;
-- Ne pas valider pour le moment.
```

Fenêtre B :

```
SELECT solde FROM compte WHERE id_compte = 105;
UPDATE compte SET solde = solde + 300 WHERE id_compte = 105;
-- La seconde instruction attend la fin de A.
```

Exécuter d'abord les requêtes de A. Dans B, exécuter le `SELECT`, puis le `UPDATE`, et observer ce qui se passe.

Retourner dans A et exécuter `ROLLBACK;`,puis Observer le comportement de B. Refaire l'expérience avec `COMMIT;`, ensuit ajout un `END;` à la fin du bloc A.

Questions : Pourquoi le `SELECT` de B fonctionne-t-il ? Pourquoi son `UPDATE` attend-il ? Que se passe-t-il lorsque on modifie A ?

Partie 3 — Déclencheurs (triggers)

Objectif

Comprendre comment un trigger réagit automatiquement aux modifications d'une table, puis créer des mécanismes de contrôle et d'historisation.

Rappel syntaxe / Exemple

Un **trigger** déclenche automatiquement une fonction lors d'un `INSERT`, `UPDATE` ou `DELETE`. Il peut intervenir `BEFORE` ou `AFTER` l'événement. Dans une fonction trigger, `NEW` représente la nouvelle ligne et `OLD` l'ancienne.

```
CREATE OR REPLACE FUNCTION annoncer_operation()
RETURNS TRIGGER AS $$

BEGIN
    RAISE NOTICE 'Operation de % a tait raliser sur le compte : %',NEW.
    type_operation, NEW.id_compte;
    RETURN NEW;
END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_annoncer_depot
AFTER INSERT ON operation
FOR EACH ROW
EXECUTE FUNCTION annoncer_depot();
```

Pour tester l'exemple, effectuer un dépôt avec la procédure de la partie 2, ou insérer une opération de test valide. Un trigger `FOR EACH ROW` est exécuté pour chaque ligne concernée.

Exercice 1 — Observer un trigger après insertion

1. Créer la fonction et le trigger présentés dans le rappel.
2. Insérer une opération de dépôt valide de 500 DA sur le compte 101 (journal de test uniquement) et observer le message.
3. Insérer une deuxième opération et les executer les deux en meme temps. Combien de fois le trigger s'exécute-t-il ?

Exercice 2 — Historiser les modifications de solde

1. Créer une fonction trigger qui insère dans `historique_solde` l'identifiant du compte, l'ancien solde et le nouveau solde.
2. Associer cette fonction à un trigger `AFTER UPDATE OF solde ON compte`, exécuté uniquement lorsque le solde a effectivement changé.
3. Modifier deux fois le solde du compte 103 et consulter l'historique.

Exercice 3 — Contrôler une modification avec BEFORE UPDATE

1. Créer une fonction trigger empêchant la diminution du solde d'un compte inactif, en utilisant `RAISE EXCEPTION`.
2. Créer le trigger `BEFORE UPDATE OF solde` associé à la table `compte`.

3. Tester une diminution de 1 000 DA sur le compte 108 (inactif), puis sur le compte 101 (actif).
4. Expliquer la différence entre un contrôle `BEFORE UPDATE` et une historisation `AFTER UPDATE`.

Exercice 4 — Combiner procédure et triggers

1. Exécuter un dépôt valide avec la procédure de la partie 2. Quels triggers sont déclenchés ?
2. Vérifier le solde, le journal `operation` et la table `historique_solde`.
3. Essayer un virement refusé. Un changement de solde ou une ligne d'historique est-il conservé ? Justifier.
4. Réflexion : quels avantages et quels risques présente l'automatisation des règles métier avec des triggers ?

Remarque

Si les procédures de la partie 2 n'ont pas été terminées, les tests peuvent être réalisés avec des `UPDATE` simples. Pour comparer les résultats entre groupes, réinitialiser le dataset avant la séance 3 ; cela supprime également les procédures précédentes, qu'il faudra recréer si nécessaire.

Bilan du TP

À l'issue de ces trois séances, vous devez savoir : (1) écrire une fonction PL/pgSQL ; (2) concevoir et appeler des procédures ; (3) expliquer `COMMIT`, `ROLLBACK`, `SAVEPOINT` et les verrous ; (4) créer et tester des triggers de contrôle et d'historisation.